UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Estructuras de Datos y Algoritmos II

Semestre: 2027-1 Grupo: 3

Proyecto 1: Sistema de Gestión y Análisis de Bugs

Implementación de Algoritmos de Ordenamiento y Búsqueda

Integrantes: Benavides Cantero Sian
González Sánchez Ayax R.
Rosas Flores Marco Saúl

Profesor: Tista García Edgar

Equipo: Equipo 1

29 de octubre de 2026

Índice

1. Objetivo **2**

2. Introducción **2**

3. Desarrollo **2**
3.1. Package modelo 2
3.2. Package algoritmos 3
3.3. Package datos 4
3.4. Package interfaz 6

4. Conclusiones **8**
4.1. Benavides Cantero Sian 8
4.2. González Sánchez Ayax R. 8
4.3. Rosas Flores Marco Saúl 8

1. Objetivo

Que el alumno realice una solución de Software implementando los algoritmos de ordenamiento más eficientes, y los algoritmos de búsqueda por comparación y transformación de llaves.

2. Introducción

En este proyecto se implementó una solución en Java que integra lo visto en clase acerca de algoritmos de ordenamiento y búsqueda, trabajando bajo restricciones que priorizan el rendimiento en memoria principal, sin persistencia en disco, sin hilos, y sin de recurrir a utilerías de ordenamiento o búsqueda nativas del lenguaje.

Para lograr un sistema robusto y mantenible, el software se modularizó en cuatro paquetes con responsabilidades desacopladas: entidad (modelo), persistencia en RAM (datos), algoritmos manuales (algoritmos) y consola interactiva (interfaz).. Destaca en este diseño la generación automática del identificador en la clase Bug, donde a partir del título y la fecha se aplica una función hash polinomial mediante la Regla de Hornerr (base 31 y módulo $10^9 + 7$), garantizando dispersión uniforme y evitando colisiones por anagramas sin desbordamientos. Este ID se vincula con la tabla hash del almacén en una estructura de lista de listas, la cual resuelve inserciones, eliminaciones y búsquedas por clave directa en tiempo constante promedio $O(1)$ mediante encadenamiento. Además, se integraron ordenamientos manuales como QuickSort por atributos y búsquedas binarias acotadas, demostrando cómo el diseño matemático y algorítmico de este programa permitió construir un sistema veloz y consistente sobre la memoria principal.

3. Desarrollo

- Explicación general de la solución de SW realizada
- Explicación de cómo se implementaron los conceptos de ordenamiento y búsqueda

3.1. Package modelo

Bug

La clase bug es la base del proyecto, pues a partir de sus elementos instanciados el resto del programa puede manipularlos y procesarlos. Además de los atributos solicitados, se inicializa un atributo tipo `long` estático constante con el valor de $10^9 + 7$, usado en la creación del ID, proceso que será explicado más adelante. Todos estos atributos son declarados como privados.

Bug tiene 2 métodos constructores, los cuales reciben como parámetros el título, prioridad, nivel de seguridad, equipo encargado, categoría y fecha de reporte del bug. El id es generado automáticamente en el primer constructor a través de su propio método "`generarID()`"; por su parte, el segundo recibe el id como parámetro. Se realizó esta separación de constructores para que en caso de una generación masiva (o no) de bugs, se ahorre tiempo del usuario o al momento de realizar pruebas.

Para generar el ID automáticamente se implementa una condensación de un string creado a partir de la concatenación del título y fecha de reporte del bug. A dicho string se le aplica una función hash polinomial, que lo transforma a un valor numérico al tratar los caracteres como coeficientes de un polinomio; en este caso se utiliza la regla de Horner, que evita llegar a potencias muy grandes al factorizar el polinomio de tal forma que solo usa sumas y multiplicaciones. Para llevarlo a cabo se recorre de izquierda a derecha el string, con una variable auxiliar h encargada de guardar los resultados parciales; se toma como base 'x' del polinomio al 31, pues además de ser un número primo, según documentación oficial de Java, en donde se utiliza el mismo método para condensar una cadena, distribuye de manera excelente los bits así reduciendo colisiones. Tras cada operación entre el 31, h y el valor del carácter, se aplica el módulo de la constante anteriormente mencionada ($10^9 + 7$), manteniendo el valor de h en un rango seguro evitando desbordamientos. Al terminar la condensación, se obtiene un número k de máximo 10 dígitos, este se concatena al prefijo "BUG-" generando así el id del bug.

Por último, se sobrescribe el método `toString` de la clase `Object` para obtener una visualización agradable y ordenada del bug con solo indicar su nombre. Se valida que su título no sea nulo, y en caso de ser muy largo, se parte para ocupar una segunda línea. También se reservan los espacios necesarios utilizando `String.format` para así visualizar los datos del bug por columnas.

Al final, se añaden los métodos setters y getters correspondientes.

3.2. Package algoritmos

Ordenar

Esta clase es en la que recae que sea posible que el usuario seleccione un criterio por el que se muestren todos los bugs, es decir, que se muestren ordenados por su prioridad, equipo encargado, fecha, etc. Además contiene el método por el cuál es posible que la lista principal se mantenga ordenada por los nombres, permitiendo que las búsquedas por nombre sean logarítmicas.

Es importante aclarar que solo existe un método que modifica la lista original (el mencionado previamente), el resto crean una copia temporal para no alterar la lista original. Los métodos de ordenamiento fueron diseñados de tal forma que no es necesario duplicar el código para cada ordenamiento diferente, es decir, tomando como referencia QuickSort, este método permite ordenar por String, según se escoja, ya sea por Título, Equipo encargado o Categoría. Es posible reutilizar el algoritmo debido a que dentro del algoritmo, existe un parámetro llamado "tipo" el cuál representa el atributo por el cuál se ordenará. ¿Cómo cambia el parámetro el funcionamiento del código? Lo que ocurre es que en todos los puntos donde se debería leer el atributo, se reemplazó por un método, el cuál según el tipo recibido, retorna el valor de un atributo diferente. Esto permite que si posteriormente se añadieran más atributos a la clase Bug, sea fácilmente añadirlos a un método de ordenamiento.

Búsqueda

En esta clase se explora la lista de bugs contenida en el almacén con el fin de encontrar coincidencias con los valores de los criterios ingresados por el usuario. Existen dos formas de

búsqueda según el o los atributos seleccionados (Sin incluir la búsqueda por transformación de claves que puede realizar `AlmacenBugs` sobre sí mismo):

- **Búsqueda por título:** Inicialmente, para realizar la consulta de bugs a través de este criterio, el método ordena el conjunto de la lista de elementos utilizando QuickSort. Posteriormente, manda a llamar a los métodos `upperBound()` y `lowerBound()` que utilizan búsqueda binaria para encontrar el índice de la primera coincidencia del título y el índice del elemento siguiente a la última coincidencia del título. Si el elemento buscado no existe dentro de la lista, los valores devueltos por estos métodos serán iguales. Finalmente, se retorna una lista que contiene todos los bugs dentro del rango generado por ambos índices.
- **Búsqueda compuesta:** Este método realiza una búsqueda lineal de los bugs del almacén que posean ciertos valores en sus atributos. Para ello, utiliza un objeto de tipo `TargetAux` que guarde los valores ingresados por el usuario y ejecuta el método `comparador()` sobre todos los bugs del almacén. Si alguno coincide con todos los valores del target, se agrega a una lista que será retornada al término de las iteraciones.

TargetAux

Esta clase posee todos los atributos de un objeto Bug ya que su objetivo es permitir que el método `busquedaCompuesta()` pueda realizar comparaciones entre los bugs almacenados y los datos ingresados por el usuario. Por consiguiente no posee un método constructor, únicamente posee setters para inicializar todos sus valores y un método `comparador()` que compara sus atributos con los de un objeto bug ingresado en su argumento, si todos coinciden retorna true, de lo contrario retorna false.

Contar

Esta clase es la que se encarga que se pueda buscar la cantidad de Bugs que coinciden con cierto criterio y mostrarlos. Para realizar esto se itera por la lista, aunque la complejidad de la iteración es $O(n)$, debido a que solo la lista original se encuentra ordenada por el título, no es posible hacer búsqueda binaria (Binary Search), ya que el costo de ordenar la lista sería de $O(n \log n)$.

3.3. Package datos

GeneradorBugs

Esta es la clase que se encargará de que se cumpla con la cantidad de bugs inicializados generados automáticamente. Está conformada principalmente por múltiples *buckets* (arreglos de tipo `String` con múltiples cadenas de texto predefinidas), de los cuales mediante la clase `Random` de Java, se seleccionan al azar varios Strings del *bucket* correspondiente. Para así generar un nuevo bug de manera inmediata. Las cadenas de texto están diseñadas con la intención de que al juntarse mantengan cierta coherencia, es decir, sea algo lógico y no parezcan varias palabras juntas de manera irracional.

La clase fue diseñada para que los *buckets* se puedan modificar de manera muy sencilla, se programó para que al iniciar el programa se detecte cuáles son todas las opciones para elegir de manera automática, es decir, en caso que se quieran eliminar o añadir elementos a los *buckets*, no es necesario modificar ninguna línea de código extra.

Para el caso de las fechas, se usa la clase `LocalDate`, y se establece como la fecha máxima el día en que el programa se ejecuta, y como fecha mínima cierta cantidad de días en el pasado (en este caso 2000), esto se diseñó así ya que simplemente se generaba un número aleatorio entre 1 y la fecha mínima y se le restaba a la fecha actual. Es importante aclarar que todos los valores mínimos y máximos, tales como las fechas o los niveles de prioridad, se definen al inicio de la clase como constantes, para que en caso que se quieran modificar sea de fácil identificación y baste con cambiar un valor y no se tenga que cambiar varias líneas del código. Es decir, diseñado para facilitar su mantenimiento.

AlmacenBugs

El almacén de bugs funciona como el repositorio donde se irá depositando cada uno de los bugs agregados; esto a través de dos estructuras que existirán simultáneamente:

Primero está un `ArrayList` inicializado del tamaño indicado por el usuario, en él se guardarán los bugs instanciados por orden de creación; esta estructura servirá como base para trabajar los algoritmos de ordenamiento y búsqueda (a excepción de la búsqueda hash). Por otro lado, está un `ArrayList` de listas ligadas, que fungirá como la tabla hash del programa, en la que cada posición del arreglo es una lista ligada a la que le corresponde una clave, que en caso de colisión, se resuelve por encadenamiento al ir agregando elementos a la lista ligada correspondiente. Como constantes auxiliares se declara una capacidad estándar para el almacén (1024) y el inverso de la constante áurea ($0.618033\ldots$) escalado al espacio de un entero en Java (2^{32}) en hexadecimal (para no mantenerlo como `int` y que sea fácilmente identificable), esto pues más adelante es usado en la función hash; se utiliza este número en específico, pues el mismo Donald Knuth demostró que es el ideal para reducir colisiones.

La función hash utilizada es el hash de Fibonacci por desplazamiento de bits. Consiste en multiplicar el valor a guardar (la parte numérica del ID del bug) por una constante, en este caso el inverso áureo anteriormente descrito, para luego desplazar el producto resultante cierta cantidad de bits hacia la derecha, obteniendo un número de n bits, que resulta en una posición específica de la tabla hash.

El almacén tiene dos constructores: El primero inicializa el almacén con la capacidad estándar, asignando un desplazamiento de bits de 22, pues al tener un entero de 32 bits, después del desplazamiento quedan los 10 bits más significativos, es decir, resulta un número del 0 al 1023.

El segundo constructor recibe una capacidad indicada por el usuario (esta debe ser obligatoriamente de base 2 para que el método funcione) para después calcular el desplazamiento correspondiente, dicho desplazamiento es equivalente a 32 menos el logaritmo base 2 de la capacidad indicada, el cual podría calcularse usando `Math.log`, pero se prefirió implementar de la siguiente manera: A partir de la capacidad entera y de base 2 indicada por el usuario, se calcula cuántos ceros consecutivos hay hacia la izquierda utilizando el método de Integer "`numberOfLeadingZeros()`" (Si la capacidad es de 512, en binario de 32 bits se representa como 22 ceros a la izquierda, un 1 en medio y 9 ceros a la derecha). Luego, dado que en Java

los 32 bits se enumeran del 0 al 31, se resta 31 − ceros encontrados anteriormente (Siguiendo con el ejemplo de 512, se hace $31 - 22 = 9$). Finalmente, se resta a 32 la cantidad anterior para conocer los bits a desplazar ($32 - 9 = 23$ bits, resultando en un número de 9 bits que va de 0 a 511).

En ambos constructores, una vez se conoce el desplazamiento de bits a utilizar, se llena el `ArrayList` correspondiente a la tabla hash con el número de listas ligadas necesarias.

En la misma clase se crean los métodos necesarios para agregar un bug, eliminarlo y buscar por ID. El primero realiza validaciones para que no se agreguen bugs nulos o con ID's repetidos, si no es el caso, se agrega a la lista de bugs y a la tabla hash en su respectiva cubeta, además de incrementar el total de bugs y retornar un `true` como señal de adición exitosa. Para eliminar bug se valida de nuevo que no sea nulo, de no ser así se ingresa directamente a la cubeta de la tabla hash a la cual corresponde el ID, y ahí se recorre la lista ligada hasta encontrarlo o no; al final se remueve el bug tanto de la lista como de la tabla hash, se decrementa el total de bugs y se retorna un `true` como señal de eliminación exitosa. Para buscar un bug, se valida que el ID ingresado no sea nulo y que tenga el sufijo "BUG-" seguido por un número de 1 a 10 dígitos, y si llega a 10, forzosamente debe iniciar con 1, esto para no sobrepasar el tamaño máximo de un entero; si el id es válido, calcula la cubeta que le corresponde con la función Hash y busca en su lista ligada alguna coincidencia de ID, si se encuentra retorna el bug.

Por último, se crea un método para poblar inicialmente de manera automática el almacén con una cantidad cualquiera de bugs. Al final se indican los setters y getters correspondientes.

3.4. Package interfaz

Menu

La clase `Menu` es la interfaz con la que el usuario final interactúa para crear y administrar el almacén de bugs. Al ejecutar el programa, el código brinda la opción de personalizar el tamaño del almacén, así como su población, utilizando el constructor secundario de `AlmacenBugs` y su método `poblarInicial()`, o crear un contenedor estándar de tamaño 1024 con 200 bugs inicializados. Para la lectura de la elección del usuario se invoca el método `confirmationReader()` y, si es el caso, se lee la cantidad de bugs iniciales con `intReader()`.

Una vez concluido el proceso anterior, se despliega un menú principal en el que el usuario puede decidir qué operación realizar. La opción seleccionada se ingresa al argumento de una estructura de control `switch` que dirige al usuario a una de las siguientes secciones:

- **Ingresar bug:** En esta sección el programa solicita al usuario los datos necesarios para crear un nuevo bug si el almacén aún posee capacidad para contenerlo. Cada lectura de datos cuida que el ingreso de valores sea concordante con los elementos aceptados por cada atributo de un objeto `Bug`; si hay alguna discrepancia, se le hace saber al usuario y se repite el escaneo de dicho valor hasta cumplir las condiciones mencionadas. Finalmente, si el ID del bug generado no existe dentro de la tabla hash de `AlmacenBugs`, se agrega al contenedor en su respectiva posición.

- **Eliminar bug:** Esta opción verifica primero que el almacén contenga al menos un bug que eliminar; de no ser así, el programa redirige al usuario al menú principal. Si existe

al menos uno, el programa solicita el ID del bug que se desea eliminar y comprueba que corresponda a un bug registrado. Antes de proceder con la eliminación, muestra en pantalla la información del bug y pide al usuario que confirme la acción. Si el usuario confirma, el bug se elimina del almacén. Si cancela la operación, puede ingresar el ID de otro bug o regresar al menú principal.

- **Buscar bugs:** Al igual que en las secciones anteriores, el programa primero verifica que el almacén no se encuentre vacío; si no lo está, accede al submenú de búsqueda. Dentro de este menú es posible realizar la búsqueda de cualquier bug existente a través de cualquiera de sus atributos. Asimismo, es posible realizar una consulta más exacta si se selecciona la opción "Compuesto", que permite ingresar varios atributos para la búsqueda de coincidencias. Cabe aclarar que dentro de esta última opción, una vez que se elige alguna característica del bug, esta deja de aparecer en los criterios de búsqueda disponibles, lo que hace más intuitivo al menú. Finalmente, tras realizar cualquier búsqueda, se imprimen los bugs que coincidieron con los valores ingresados. En esta parte se invocan los métodos de búsqueda por transformación de claves (de la clase `AlmacenBugs`), búsqueda binaria por título y búsqueda compuesta (ambas de la clase `Busqueda`).

- **Listar bugs:** Aquí, el usuario puede elegir imprimir de forma ordenada los bugs contenidos en el almacén a partir de cualquiera de los atributos de un objeto bug. Para las variables de instancia numéricas, el ordenamiento se realiza de forma ascendente. Para las variables de instancia de tipo `String`, el ordenamiento se realiza de forma lexicográfica. En esta parte se utilizan los algoritmos QuickSort, Counting Sort y MergeSort.

- **Conteo de bugs:** Este submenú permite al usuario contabilizar y visualizar los bugs que compartan un cierto atributo en común. Dado que únicamente los ID son elementos no repetibles, el conteo puede realizarse con base en cualquier otra variable de instancia de los bugs.

Read

En esta clase se incluyen métodos auxiliares útiles en la lectura de valores ingresados por el usuario. Cada uno previene algún tipo de excepción (exceptuando `confirmationReader()`) producida por un valor incompatible ingresado por el usuario al momento de solicitar algún tipo de información específica:

- `intReader()`: Realiza la lectura de un dato ingresado por el usuario hasta que este sea un valor numérico de tipo entero. La excepción que previene es `NumberFormatException`, la cual se deriva de invocar el método `parseInt()` de la clase `Integer` sobre un dato que no es una cadena que represente en su totalidad a un número entero.

- `floatReader()`: Realiza la lectura de un dato ingresado por el usuario hasta que este sea un valor numérico de tipo flotante. La excepción que previene es `NumberFormatException`, la cual se deriva de invocar el método `parseFloat()` de la clase `Float` sobre un dato

que no es completamente una cadena que represente en su totalidad a un número de punto flotante.

- `dateReader()`: Realiza la lectura de un dato ingresado por el usuario hasta que este tenga el formato de una fecha `YYYY-MM-DD`. De igual forma, tampoco permite fechas incongruentes (como 30 de febrero). La excepción que previene es `DateTimeParseException`, la cual surge cuando se invoca el método `parse()` de la clase `LocalDate` sobre un dato que no cumple con las condiciones antes mencionadas.
- `confirmationReader()`: A diferencia de los métodos anteriores, este no previene alguna excepción mediante conversiones numéricas o de fechas, sino que su función radica en verificar si el usuario ingresó un carácter aceptable al momento de confirmar o negar alguna acción dentro del programa (`'Y'`, `'y'`, `'N'`, `'n'`). Esto reduce el acoplamiento y facilita la modificación de los valores aceptables para estas situaciones.

4. Conclusiones

4.1. Benavides Cantero Sian

El proyecto permitió profundizar en el comportamiento y la implementación de los algoritmos de búsqueda y ordenamiento más conocidos, aplicados en el diseño de una solución de software real. En este sentido, las diversas necesidades de la propuesta elegida no siempre requirieron el mismo algoritmo. Un ejemplo de ello es la impresión ordenada de los bugs a partir de ciertos atributos: para ordenar por título o por equipo encargado, el algoritmo más conveniente fue Quicksort, mientras que para ordenar por nivel de seguridad o prioridad resultó más adecuado Counting Sort, debido a que su funcionamiento se basa en el conteo de valores dentro de un rango reducido. Esto exigió analizar cada herramienta disponible durante la planificación del programa y de sus módulos. Asimismo, la organización del código en distintos paquetes, aunque no se relaciona directamente con el objetivo principal del proyecto, pero sí con el paradigma utilizado durante el curso, resultó fundamental para dar claridad a cada parte desarrollada. Por lo anterior, los objetivos del proyecto se cumplieron.

4.2. González Sánchez Ayax R.

Considero que los objetivos del proyecto se cumplieron debido a que la implementación del proyecto elegido implementa algoritmos de ordenamiento y búsqueda tanto por comparación de llaves como por transformación de llaves. Durante el desarrollo del proyecto se puso a prueba la capacidad de pensar la forma óptima de resolver los requerimientos solicitados. Además de ello, se puso en práctica varios conceptos de programación orientada a objetos, debido a los cuales fue posible realizar el proyecto de manera efectiva. Ya que aunque no era el tema central, fue una parte fundamental en el desarrollo.

4.3. Rosas Flores Marco Saúl

La realización de este proyecto me permitió utilizar herramientas muy importantes y útiles para cualquier desarrollo de software, como lo son github y lucidchart. Con ellos, se logró

tener una buena organización y planeación con el equipo, dándonos una base sólida para llevar a cabo el programa. En él, tuvimos siempre en cuenta los recursos utilizados, tanto de tiempo como de memoria, a la hora de implementar cualquier componente del código, llegando a optimizar el sistema tanto como las restricciones nos lo permitieron. Sin embargo, con ello pude darme cuenta de que a pesar de lograr una búsqueda en tiempo constante promedio $O(1)$ a través de la tabla hash, no se puede prescindir por completo de los demás tipos de algoritmos de ordenamiento y búsqueda, pues como se pudo observar, son necesarios para la búsqueda personalizada por atributos o para visualizar un listado de los bugs en el sistema ordenado bajo cierto criterio. En conclusión, en un sistema de gestión de cualquier tipo de elementos, los distintos algoritmos vistos en clase se complementan entre sí logrando un software ágil y útil para diversas tareas.